Modernizing JackTrip: Two New Hub Client Implementations

A.I. Assistant

Networked Music Performance

August 27, 2025

What is JackTrip?

- A system for high-quality, low-latency audio streaming over the internet.
- Designed specifically for networked music performance, enabling musicians to play together in real-time from different locations.
- Core technology relies on raw UDP packets for minimal overhead.
- Often used in a "Hub Server" model, where multiple clients connect to a central server that mixes and redistributes the audio.

The Challenge: Building Modern Clients

The JackTrip hub server is a robust, command-line driven system. How can we build clients that are more accessible and integrate with modern platforms?

We will explore two distinct approaches:

1. A native desktop application using Qt6/C++.
2. A web-based application using the Web Audio API and a proxy.

Core Prompts & Citations

Prompt 1: Qt6/C++ Client

You are a careful, concise assistant. Don't give commentary or anthropomorphize. Be very attentive to prompt continuations because they often create a single character syntax problem. The purpose is to make a qt6/c++ application that implements a JackTrip client that can connect to a running JackTrip hub server. It should connect with bi-directional audio and exchange JackTrip audio packets. Sample rate is 48000, bit depth is 16, channel count is 2, server IP address is 171.64.197.158. The initial handshake should initiate the connection on the canonical port and receive an ephemeral UDP port back from the server. This is described in the article uploaded. Also uploaded is an older working client but we want to modernize and use Qt6 audio.

Prompt 2: Web App Conversion

convert this working c++ project to a single-page web audio webapp using the most appropriate web udp streaming sockets

Prompt 3: WebRTC Postlude

the presentation is ready, now the kicker, make a postlude in the presentation that accounts for the content of the webRTC article I've attached

Citations

- C. Chafe, "I am Streaming in a Room" Frontiers in Digital Humanities, vol. 5, 2018. DOI: 10.3389/fdigh.2018.00027
- `https://github.com/cchafe/hacktrip`: jacktrip hub client with rtaudio and library API.
- M. Sacchetto, P. Gastaldi, C. Chafe, C. Rottondi, and A. Servetti, "Web-Based Networked Music Performances via WebRTC: A Low-Latency PCM Audio Solution" J. Audio Eng. Soc., vol. 70, no. 11, pp. 926–937, (2022 November). DOI: 10.17743/jaes.2022.0021

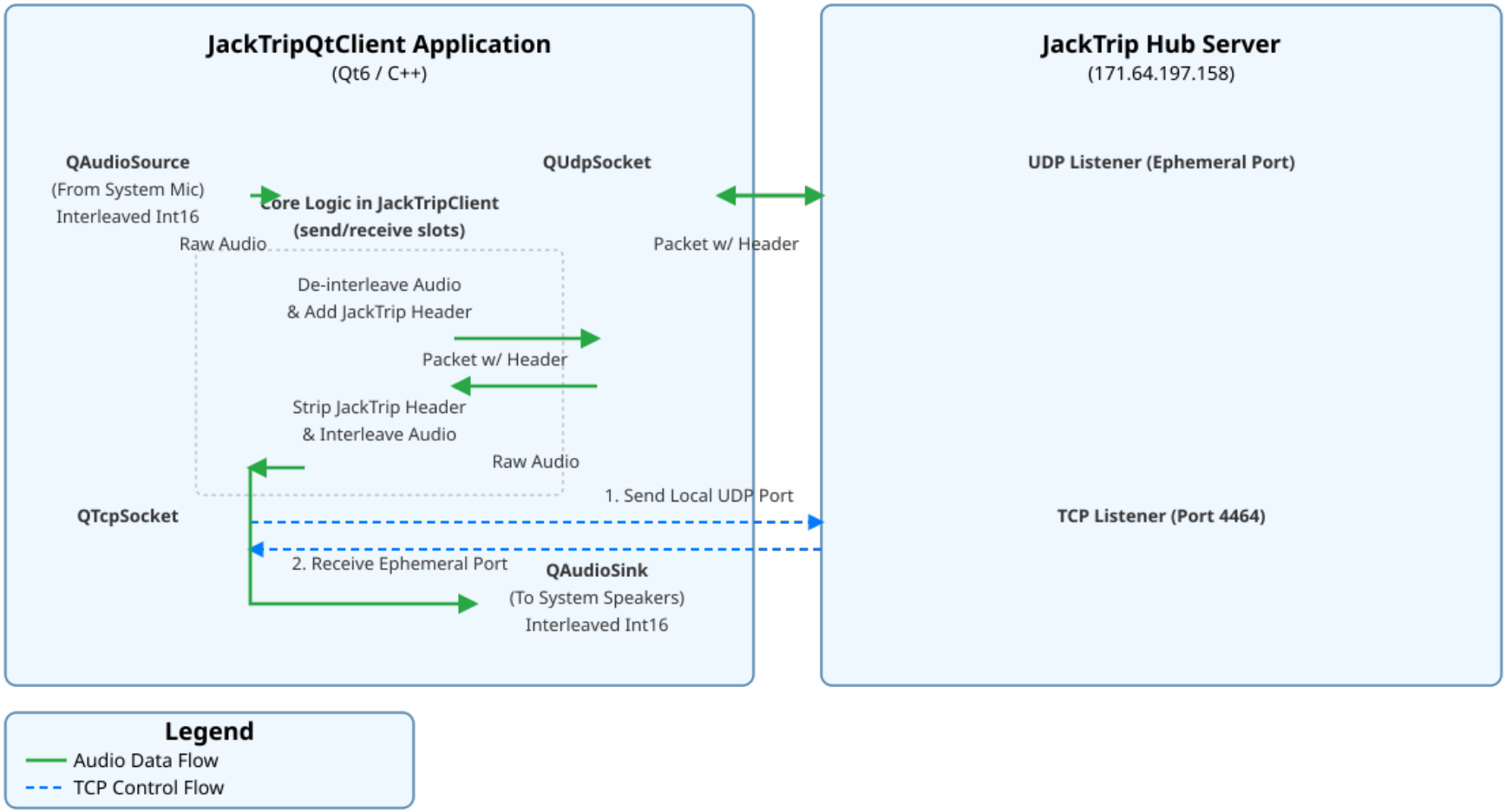
Development Journey: A Log of Prompts

This project evolved through an iterative process of prompts and refinements:

- **Initial Goal:** Create a Qt6/C++ JackTrip client.
- **Refinement:** Implement non-interleaved audio conversion.
- **Pivot:** Convert the C++ project to a single-page web audio app.
- **Debug 1:** Resolve "Unable to load a worklet's module" by embedding the script.
- **Debug 2:** Correct the TCP handshake byte order for the proxy.
- **Feature:** Add the required JackTrip header to audio packets in the proxy.
- **Debug 3:** Fix a 'RangeError' by ensuring the sequence number wraps around.
- **Documentation:** Generate block diagrams (SVG, then PNG) and this LaTeX presentation.
- **Final Polish:** Refine LaTeX syntax highlighting and add the WebRTC postlude.

Architecture: Qt6/C++ Client

This approach uses Qt's rich libraries to create a self-contained, high-performance native application.



Key Logic: Sending Audio Data

The client performs two key steps before sending audio: de-interleaving the audio channels and prepending the JackTrip header.

```
1  void JackTripClient::sendAudioData()
2  {
3      // 1. Read interleaved audio from QAudioSource
4      QByteArray interleavedData = m_audioInputDevice->read(...);
5
6      // 2. Convert to non-interleaved format for the server
7      QByteArray nonInterleavedData(...);
8      // ... de-interleaving logic ...
9
10     // 3. Prepare and send packet with header
11     QByteArray packet;
12     m_header.SeqNumber = qToLittleEndian(m_sequenceNumber++);
13     m_header.TimeStamp = qToLittleEndian(...);
14
15     packet.append(reinterpret_cast<const char*>(&m_header), ...);
16     packet.append(nonInterleavedData);
17
18     m_udpSocket->writeDatagram(packet, ...);
19 }
```

Qt6/C++ Client: Pros and Cons

Advantages

- High Performance: Direct access to system resources and low-level networking.
- Low Latency: Minimal overhead between audio hardware and the network socket.
- Self-Contained: All logic in one executable.

Disadvantages

- Complex Deployment: Requires compilation for each target platform (Windows, macOS, Linux).
- Less Accessible: Users must download and install a specific application.
- Steeper Learning Curve: Requires C++ and Qt knowledge.

Qt6/C++ Client: Pros and Cons

Advantages

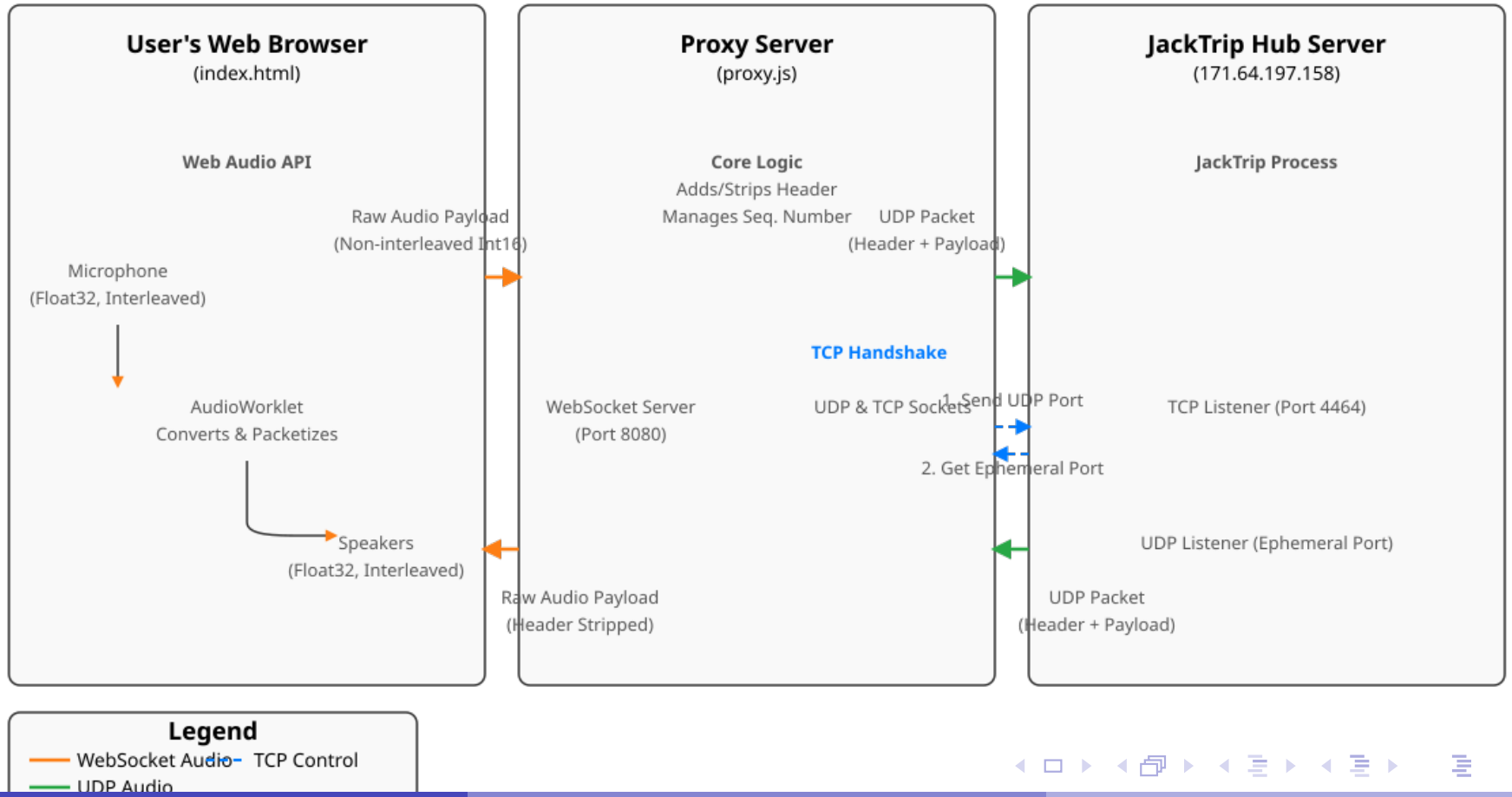
- High Performance: Direct access to system resources and low-level networking.
- Low Latency: Minimal overhead between audio hardware and the network socket.
- Self-Contained: All logic in one executable.

Disadvantages

- Complex Deployment: Requires compilation for each target platform (Windows, macOS, Linux).
- Less Accessible: Users must download and install a specific application.
- Steeper Learning Curve: Requires C++ and Qt knowledge.

Architecture: Web Audio + Proxy

This approach uses a standard web browser for the UI and a lightweight Node.js proxy to bridge the gap between web technologies and the JackTrip protocol.



The Proxy's Crucial Role

- Web browsers cannot create raw TCP or UDP sockets for security reasons.
- The proxy server acts as a translator:
  - Speaks WebSockets to the browser.
  - Speaks raw UDP/TCP to the JackTrip server.
- It handles the entire JackTrip protocol, including the initial handshake and adding/stripping the header from every audio packet.

Key Logic: The Proxy Header

The Node.js proxy adds the header before forwarding audio to the JackTrip server.

```
ws.on('message', audioPayload => {
    // Create a buffer for the full packet
    const packet = Buffer.alloc(HEADER_SIZE + audioPayload.length);

    // 1. Write the JackTrip header (all Little Endian)
    packet.writeBigUInt64LE(BigInt(Date.now()), 0);
    packet.writeUInt16LE(sequenceNumber, 8);
    // ... write other header fields ...

    // Increment and wrap the sequence number
    sequenceNumber = (sequenceNumber + 1) & 0xFFFF;

    // 2. Copy the audio payload from the web client
    audioPayload.copy(packet, HEADER_SIZE);

    // 3. Send the complete packet via UDP
    udpSocket.send(packet, ...);
});
```

Web Audio Client: Pros and Cons

Advantages

- Maximum Accessibility: Runs in any modern web browser. No installation needed.
- Easy Deployment: Simply host the HTML and JS files on a web server.
- Rapid Development: Faster UI development with web technologies.

Disadvantages

- Higher Latency: Audio passes through more layers (Browser -¿ WebSocket -¿ Proxy -¿ UDP).
- Complex Setup: Requires a separate proxy server to be running.
- Browser Limitations: Dependent on Web Audio API performance and security policies.

Web Audio Client: Pros and Cons

Advantages

- Maximum Accessibility: Runs in any modern web browser. No installation needed.
- Easy Deployment: Simply host the HTML and JS files on a web server.
- Rapid Development: Faster UI development with web technologies.

Disadvantages

- Higher Latency: Audio passes through more layers (Browser -¿ WebSocket -¿ Proxy -¿ UDP).
- Complex Setup: Requires a separate proxy server to be running.
- Browser Limitations: Dependent on Web Audio API performance and security policies.

Side-by-Side Comparison

Aspect	Qt6 / C++ Client	Web Audio + Proxy
Latency	Lowest	Higher
Accessibility	Low (Requires install)	Highest (Browser-based)
Deployment	Per-platform binaries	Host web files, run proxy
Complexity	All in one app	Two-part system
Best For	Pro users, studios	Casual users, education

Conclusion

- We have successfully implemented two modern clients for the classic JackTrip hub server.
- The **Qt6/C++** approach offers the highest performance and is ideal for professional, latency-critical applications.
- The **Web Audio** approach provides unparalleled accessibility and ease of use, opening up networked music performance to a much wider audience.
- Both models demonstrate the flexibility of the underlying JackTrip protocol and provide a path forward for its continued use and evolution.

A More Direct Path: WebRTC

The WebSocket+Proxy model works, but it has an inherent limitation: the proxy itself adds latency and is an extra point of failure.

Question: Can we connect the browser *directly* to a JackTrip-like server?

Answer: Yes, by leveraging WebRTC.

However, standard WebRTC (used for video calls) is unsuitable for music, as it applies audio compression and processing (echo cancellation, etc.) that distort the signal. The attached article provides a solution.

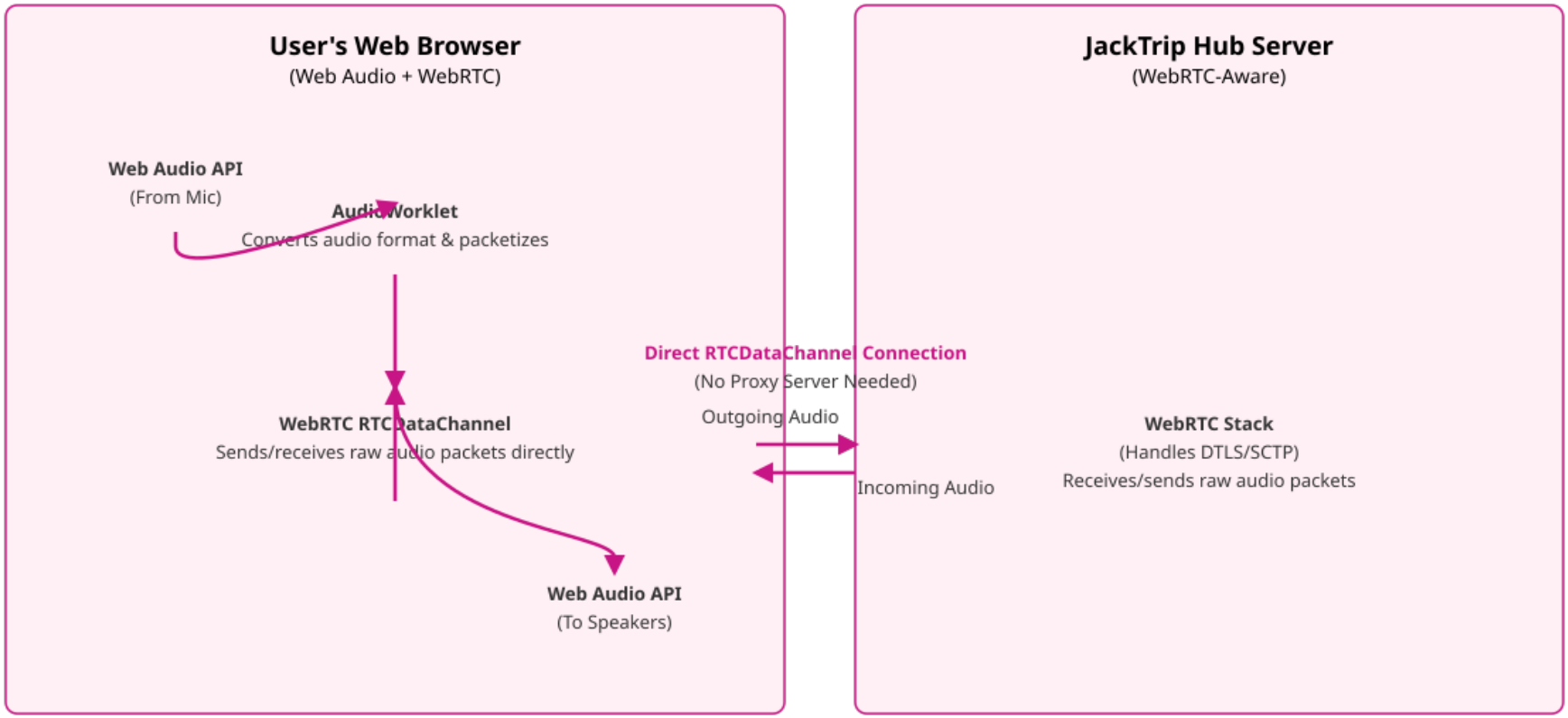
The RTCDataChannel Solution

The key insight from the research is to bypass WebRTC's media-processing pipeline and use the **RTCDataChannel**.

- The 'RTCDataChannel' is designed to send arbitrary binary data.
- This allows us to use the same architecture as our web client:
  - Use 'Web Audio' and an 'AudioWorklet' to capture and packetize raw, non-interleaved audio.
  - Send these raw packets directly over the 'RTCDataChannel'.
- This achieves a direct, low-latency connection from the browser to a server without unwanted audio processing.

New Architecture: The Proxy-less Client

This refined architecture eliminates the middleman, simplifying the system and reducing latency.



The Future: WebRTC vs. WebSocket

Aspect	**WebSocket + Proxy (Current)**	**WebRTC DataChannel (Future)**
Architecture	3-Tier (Browser-Proxy-Server)	2-Tier (Browser-Server)
Latency	Good, but with proxy overhead	Potentially Lower
Server	**Standard JackTrip Server** (via a required proxy)	Must be modified to be a WebRTC endpoint (handle DTLS/SCTP)
Overall	Highly accessible, proven to work	The ideal, most performant architecture for web-based clients

The Future: WebRTC vs. WebSocket

Aspect	**WebSocket + Proxy (Current)**	**WebRTC DataChannel (Future)**
Architecture	3-Tier (Browser-Proxy-Server)	2-Tier (Browser-Server)
Latency	Good, but with proxy overhead	Potentially Lower
Server	**Standard JackTrip Server** (via a required proxy)	Must be modified to be a WebRTC endpoint (handle DTLS/SCTP)
Overall	Highly accessible, proven to work	The ideal, most performant architecture for web-based clients

Thank You

Questions?